Day 84 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 13, Day 1 reference

Watch: Byte Pair Encoding Tokenization (Hugging Face) | Let's build the GPT Tokenizer (Andrej Karpathy)

`day84_byte_pair_encoding.py`, segment by segment, in the order `main()` uses things. Day 62's reader is copied verbatim and explained there. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: a word = a tuple of symbols ending in </w> · vocab = word (as symbols) → how often it occurs · a merge = a pair (a, b) that becomes the symbol ab · rank = when a merge was learned (0 = first)

Segment 0 — Setup

Settings

CODE

```
EOW = "</w>"
N_MERGES = 4000
SWEEP = (0, 100, 250, 500, 1000, 2000, 4000)
```

- `EOW = "</w>"` — a string that cannot occur inside a word of the text, used as the end-of-word symbol.
- `SWEEP` — the merge counts compared in Section 4; all are prefixes of one 4000-merge run.

Segment 1 — BPE

pair_counts

MATH

$$count(a, b) = \Sigma_{words\ w}\ freq(w) \times (times\ a\ b\ is\ adjacent\ in\ w)$$

CODE

```
def pair_counts(vocab):
    counts = Counter()
    for symbols, freq in vocab.items():
        for a, b in zip(symbols, symbols[1:]):
            counts[a, b] += freq
    return counts
```

- `Counter()` — a dictionary that starts every missing key at 0.
- `zip(symbols, symbols[1:])` — adjacent pairs: each symbol with the one after it.
- `counts[a, b] += freq` — a tuple key written without brackets; every occurrence of the word counts.

merge_word

CODE

```
def merge_word(symbols, pair, new):
    out, i = [], 0
    while i < len(symbols):
        if i + 1 < len(symbols) and (symbols[i], symbols[i + 1]) == pair:
            out.append(new); i += 2
        else:
            out.append(symbols[i]); i += 1
    return tuple(out)
```

- `while i < len(symbols)` — a while loop: i jumps by 2 after a merge, by 1 otherwise.
- `(symbols[i], symbols[i + 1]) == pair` — tuples compare element by element.
- `return tuple(out)` — tuples are immutable, so they can be dictionary keys (the vocab).

train_bpe

MATH

$$(a, b) = \text{argmax count}(a, b); \quad \text{every } a\ b \to ab; \quad \text{repeat}$$

CODE

```
def train_bpe(word_counts, n_merges):
    vocab = {tuple(w) + (EOW,): c for w, c in word_counts.items()}
    merges = []
    for _ in range(n_merges):
        counts = pair_counts(vocab)
        if not counts:
            break
        best = max(counts, key=lambda p: (counts[p], p))
        new = best[0] + best[1]
        vocab = {merge_word(s, best, new): c for s, c in vocab.items()}
        merges.append((best, counts[best]))
    return merges
```

- `tuple(w) + (EOW,)` — a word's characters as a tuple, plus a one-element tuple: the comma makes it a tuple.
- `max(counts, key=lambda p: (counts[p], p))` — largest count first; on a tie the larger pair, so every run is the same.
- `{merge_word(s, best, new): c for s, c in vocab.items()}` — a dictionary comprehension rebuilds the vocab with the merge applied.

encode_word, decode and ranks_of

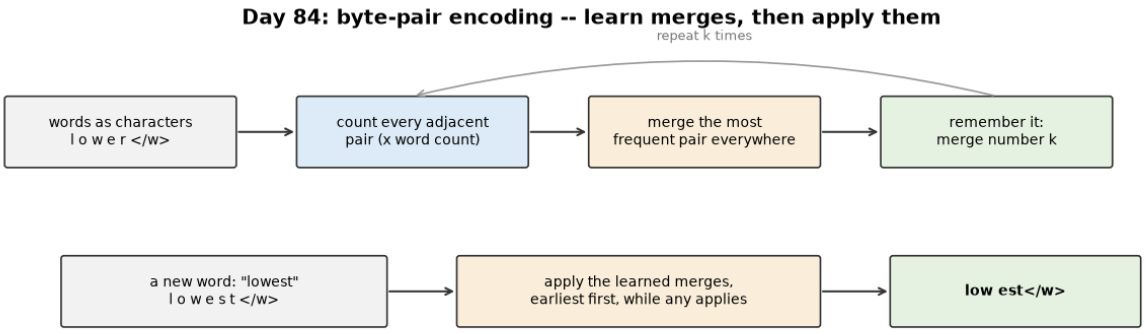
CODE

```
def encode_word(word, ranks):
    symbols = tuple(word) + (EOW,)
    while len(symbols) > 1:
        pairs = [(ranks.get(p, np.inf), p) for p in zip(symbols, symbols[1:])]
        rank, best = min(pairs)
        if rank == np.inf:
            break
        symbols = merge_word(symbols, best, best[0] + best[1])
    return symbols
def decode(symbols):
    return "".join(symbols).replace(EOW, " ").strip()
def ranks_of(merges, k):
    return {pair: i for i, (pair, _) in enumerate(merges[:k])}
```

- `ranks.get(p, np.inf)` — the merge's rank, or infinity for a pair never learned.

- `rank, best = min(pairs)` — tuples compare by their first item: the earliest-learned pair present.
- `"".join(symbols).replace(EOW, " ")` — glue every symbol together; each end mark becomes a space.
- `enumerate(merges[:k])` — the first k merges, with their positions.

SYSTEM DIAGRAM



Learning merges, then applying them.

Segment 2 — Experiments

demo_lossless: encode and decode every test sentence

CODE

```
for sent in test_sents:
    words = [w for w, _ in sent]
    symbols = [s for w in words for s in encode_word(w, ranks)]
    wrong += decode(symbols) != " ".join(words)
    n_tok += len(symbols); n_words += len(words)
```

- `[s for w in words for s in encode_word(w, ranks)]` — a nested comprehension: every subword of every word, in order.
- `wrong += decode(symbols) != ...` — True counts as 1: the number of sentences that do not come back exactly.

demo_unknown: characters never seen

CODE

```
new_chars = sorted({ch for w in test_words for ch in w} - set(chars))
for ch in new_chars:
    print(f"  U+{ord(ch):04X}: {len(ch.encode('utf-8'))} byte(s) in UTF-8; in {sum(ch in w for w
        in test_words)} test word(s)")
```

- `{...} - set(chars)` — set difference: characters in the test text that training never had.
- `f"U+{ord(ch):04X}"` — ord gives the character's Unicode number; :04X prints it as 4 hexadecimal digits.
- `ch.encode('utf-8')` — the character's bytes in UTF-8: 1 to 4 of them.

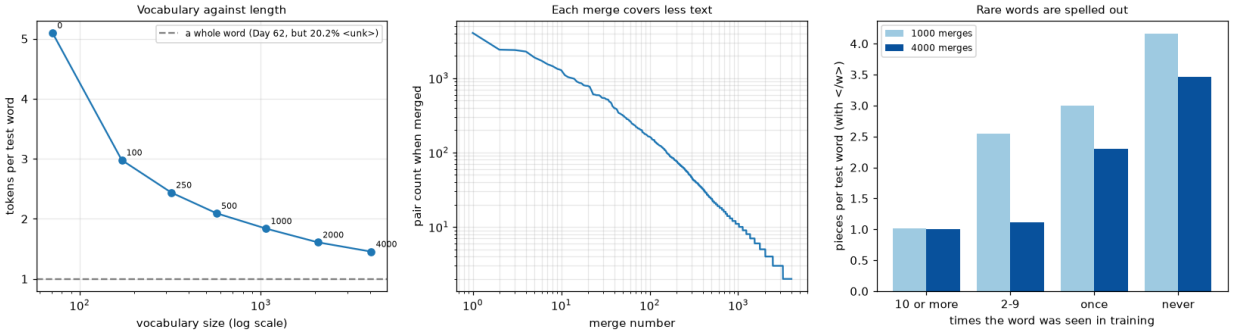
demo_frequency: lambdas as tests

CODE

```
bands = [("10 or more", lambda c: c >= 10), ("2-9", lambda c: 2 <= c <= 9), ("once", lambda c: c
    == 1),
         ("never", lambda c: c == 0)]
```

- `lambda c: 2 <= c <= 9` — a chained comparison inside a one-line function.
- `counts[w]` — a Counter returns 0 for a word it never saw, so 'never' needs no special case.

REAL GRAPH



Real graph from the script: vocabulary against length, merge counts, pieces by frequency.

Quick Syntax Glossary — new or reinforced constructs

- `Counter()` — counts with a default of 0.
- `zip(s, s[1:])` — adjacent pairs.
- `(x,)` — a one-element tuple.
- `max(d, key=lambda k: (d[k], k))` — the largest value, ties broken by the key.
- `dict.get(k, default)` — a value, or a default.
- `set_a - set_b` — set difference.
- `ord(ch), f"{n:04X}"` — a character's number, printed in hexadecimal.
- `s.encode("utf-8")` — text to bytes.

End of Day 84 syntax reference. For the experiments and what they showed, see the Day 84 complete guide.